

TypeScript Utility Types

Die wichtigsten eingebauten Hilfsklassen mit Praxisbeispielen

TypeScript bietet mächtige Werkzeuge, um bestehende Typen zu transformieren, ohne Code zu duplizieren. Hier sind die 6 am häufigsten verwendeten Utility Types im Überblick:

1. Pick<Type, Keys>

Wählt gezielt eine Untermenge von Eigenschaften ($K \subseteq T$) aus einem Typen aus.

```
interface User {
  id: string;
  name: string;
  email: string;
  isAdmin: boolean;
}

// Erstellt einen Typen mit NUR 'id' und 'email'
type UserContactInfo = Pick<User, "id" | "email">;

const contact: UserContactInfo = {
  id: "usr_123",
  email: "max@mustermann.de"
};
```

2. Omit<Type, Keys>

Das Gegenteil von Pick: Erstellt einen Typen, indem alle Eigenschaften außer den angegebenen Schlüsseln kopiert werden.

```
// Erstellt einen Typen ohne die 'id' (z.B. für Neuanlagen vor dem DB-Insert)
type NewUser = Omit<User, "id">;

const candidate: NewUser = {
  name: "Max",
  email: "max@mustermann.de",
  isAdmin: false
};
```

3. Partial<Type>

Schaltet alle Eigenschaften eines Typs auf *optional* (?). Perfekt für Update-Payloads.

```
// Alle Felder sind jetzt optional (z.B. { name?: string; ... })
type UpdateUserData = Partial<User>;

function updateUser(changes: UpdateUserData) {
  // Nimmt z.B. nur { isAdmin: true } entgegen
}
```

4. Required<Type>

Das genaue Gegenteil von Partial. Es entfernt alle optionalen Modifikatoren und erzwingt jede Eigenschaft.

```
interface Config {
  title?: string;
  timeout?: number;
}

// Macht alle Eigenschaften zu Pflichtfeldern
type StrictConfig = Required<Config>;

const appConfig: StrictConfig = {
  title: "Mein System",
  timeout: 5000 // Darf nicht fehlen!
};
```

5. Readonly<Type>

Setzt alle Eigenschaften auf *schreibgeschützt*. Nachträgliche Zuweisungen führen zu einem Compiler-Fehler.

```
const settings: Readonly<{ env: string }> = {
  env: "production"
};

// settings.env = "staging"; // FEHLER: Eigenschaft ist schreibgeschützt!
```

6. Record<Keys, Type>

Erstellt ein strukturiertes Objekt-Dictionary, bei dem die Keys und der Wert-Typ vordefiniert sind.

```

type Role = "admin" | "editor" | "viewer";

// Weißt jeder Rolle ein Array von Berechtigungs-Strings zu
const permissions: Record<Role, string[]> = {
  admin: ["read", "write", "delete"],
  editor: ["read", "write"],
  viewer: ["read"]
};

```

Utility Type	Kurzbeschreibung
Pick<T, K>	Wählt nur ausgewählte Eigenschaften aus.
Omit<T, K>	Entfernt spezifizierte Eigenschaften.
Partial<T>	Macht alle Eigenschaften optional.
Required<T>	Macht alle Eigenschaften zu Pflichtfeldern.
Readonly<T>	Verhindert nachträgliche Wertänderungen.
Record<K, T>	Erzeugt ein typisiertes Key-Value Mapping.